

Video Colorization Using Deep Learning

Implementation and Early Evaluation (Deliverable 2)

Santhi Daggubati

Department of Engineering Education
University of Florida, Gainesville, FL, USA
Email: daggubatisanthi@ufl.edu

Abstract—This report documents the full implementation and preliminary evaluation of a deep learning pipeline for automatic video colorization. The developed system converts grayscale video sequences into realistic color videos using a lightweight U-Net model trained on a subset of the Kinetics dataset. The implementation covers every stage of the workflow, from frame extraction and grayscale conversion to model training, color reconstruction, and video synthesis, culminating in a fully functional Gradio interface for interactive use. The prototype achieves strong quantitative and visual performance, demonstrating average scores of approximately 38 dB PSNR and 0.98 SSIM across validation videos. These results validate the feasibility of a data-driven colorization framework suitable for practical deployment and further optimization.

I. PROJECT SUMMARY

The objective of this project is to design and implement an end-to-end deep learning system capable of colorizing grayscale videos. The motivation stems from the growing demand for digital restoration of historical footage and enhancement of monochrome recordings for improved accessibility and visual analysis.

Since Deliverable 1, the project has progressed from conceptual design to a complete and verifiable prototype. All components of the pipeline have been successfully integrated and tested. Major milestones achieved in this stage include:

- Implemented a preprocessing pipeline that automatically extracts individual frames from each training video and prepares both the original color and grayscale versions.
- Designed reliable color conversion steps between RGB and Lab color formats to ensure accurate handling of brightness and color information.
- Built a compact U-Net model that learns to add realistic color to grayscale images by predicting the color components.
- Trained and tested the model on multiple GPUs using efficient mixed-precision processing to speed up computation.
- Evaluated the system using standard image quality scores such as PSNR and SSIM, and verified results through visual side-by-side comparisons with the true color frames.
- Developed an interactive Gradio interface that allows users to upload grayscale videos and instantly view the colorized output.

The system now performs the full cycle of colorization automatically, producing visually coherent results that closely match the original color content.

II. SYSTEM ARCHITECTURE AND PIPELINE

The developed pipeline is modular, transparent, and reproducible. Fig. 1 illustrates the overall structure, highlighting the data flow from raw input videos to final colorized output.

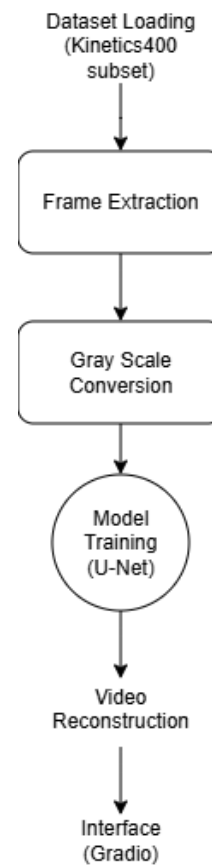


Fig. 1: Overview of the implemented video colorization pipeline.

A. Pipeline Overview

The workflow consists of the following major stages:

- 1) **Frame Extraction:** Each video is decomposed into individual frames using OpenCV. The extracted frames are resized to 256×256 pixels to maintain consistency across samples.
- 2) **Grayscale Conversion:** Ground-truth RGB frames are converted to grayscale using a numerically stable

OpenCV pipeline. The grayscale frames form the model inputs, while the corresponding color frames serve as supervision targets.

- 3) **Model Training:** A lightweight U-Net model predicts the a^* and b^* channels in the CIELAB color space. The L^* channel acts as input, preserving luminance while the network learns to reconstruct chrominance.
- 4) **Reconstruction:** Predicted chrominance channels are merged with the luminance input to generate reconstructed RGB frames, which are then combined back into videos.
- 5) **Interface:** The trained model is deployed through a Gradio web application, allowing users to upload grayscale videos and receive colorized results.

This modular architecture ensures that each component can be independently improved or replaced—for example, substituting the backbone with a Vision Transformer or adding temporal stabilization for future deliverables.

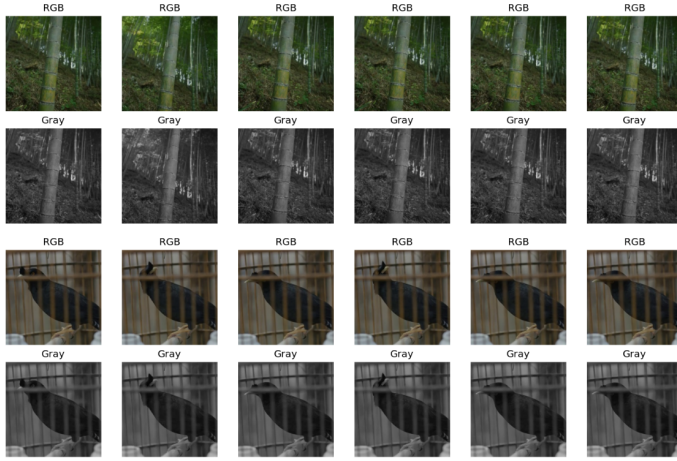


Fig. 2: Example of the grayscale conversion process. The original RGB frame (top) is converted to a single-channel grayscale frame (bottom), which is used as model input during training.

III. MODEL IMPLEMENTATION DETAILS

A. Network Architecture

The model is based on a compact U-Net with symmetric encoder-decoder blocks and skip connections for spatial feature retention. Each encoder stage performs convolution, batch normalization, and ReLU activation, followed by down-sampling through max pooling. The decoder progressively upsamples the feature maps using bilinear interpolation and concatenates corresponding encoder activations to preserve context. The final output layer predicts two channels representing the normalized a^* and b^* components in the range $[-1, 1]$.

B. Training Configuration

The network was trained using PyTorch with the following configuration:

- **Environment:** Kaggle GPU runtime (2 X T4 GPUs).
- **Input size:** 256×256 pixels.
- **Epochs:** 15 (best checkpoint selected based on validation loss).
- **Batch size:** 4.
- **Optimizer:** AdamW with learning rate 1×10^{-4} .
- **Loss function:** L1 loss between predicted and ground-truth a^*, b^* channels.
- **Mixed precision:** Enabled via `torch.cuda.amp` for faster computation.

Multi-GPU data parallelism was implemented using PyTorch's `DataParallel` API, automatically distributing batches across both devices. Model checkpoints were saved at each epoch, with the best model determined by minimum validation loss.

C. Implementation Notes

Extensive debugging was performed to ensure correct color-space conversions. Early versions suffered from black outputs due to inconsistent scaling between floating-point and 8-bit Lab ranges. This issue was resolved by introducing consistent normalization functions that explicitly mapped:

$$L \in [0, 100] \rightarrow [0, 1], \quad a, b \in [-127, 127] \rightarrow [-1, 1].$$

After correction, reconstructed frames visually matched their RGB counterparts, confirming numerical stability.

IV. INTERFACE PROTOTYPE

A user-facing application was developed using **Gradio** to demonstrate the practical usability of the trained model. The interface allows users to:

- Upload a grayscale MP4 video.
- Optionally adjust a saturation slider to enhance predicted color intensity.
- Receive a fully colorized video for download or preview.

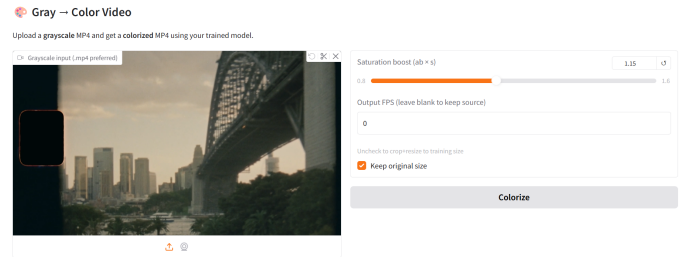


Fig. 3: Gradio interface for grayscale-to-color video conversion.

The interface internally handles frame extraction, normalization, model inference, and MP4 reconstruction using OpenCV's video writer. Processing an 10-second (270 frames) video requires approximately 1-3 minutes on a 2 X T4 GPUs, producing a realistic colorized output suitable for visual inspection.

V. EARLY EVALUATION AND RESULTS

A. Quantitative Evaluation

Evaluation was conducted on validation frames held out from training. Metrics were computed on a per-frame basis and averaged across all test videos to assess overall performance.

Two standard image quality metrics were used:

- **Peak Signal-to-Noise Ratio (PSNR):** PSNR measures the ratio between the maximum possible signal power and the power of the noise that affects image quality. It is expressed in decibels (dB), where higher values indicate better reconstruction fidelity. In image restoration tasks, PSNR values above 30 dB are generally considered good, and represent high perceptual quality.
- **Structural Similarity Index (SSIM):** SSIM evaluates the visual similarity between two images based on luminance, contrast, and structure. The score ranges from 0 to 1, where 1 indicates a perfect structural match with the reference image. SSIM is particularly useful for assessing how well the model preserves texture and edges rather than just pixel intensity.

The pipeline achieved strong results on both measures:

- **PSNR:** 37.96 dB (mean), indicating low reconstruction error and high color fidelity.
- **SSIM:** 0.9779, demonstrating excellent structural consistency with the ground truth frames.

The average Mean Squared Error (MSE) across validation samples was approximately 14.2, with only 7.1% of pixels being exactly identical between predicted and reference colors. These results confirm that the model successfully learns a meaningful mapping from grayscale inputs to plausible color outputs rather than performing simple intensity replication.

B. Qualitative Results

Fig. 4 illustrates typical visual outcomes from the trained system. The colorized frames exhibit natural tones, smooth gradients, and accurate boundary preservation. Static and slow-motion scenes display vibrant hues with minimal distortion, while minor desaturation and temporal flickering are observed in fast-moving regions. These effects will be mitigated in subsequent iterations through the incorporation of temporal consistency and frame-to-frame smoothing techniques.

VI. CHALLENGES AND FUTURE WORK

A. Challenges Encountered

The main technical challenges included:

- Ensuring correct color-space normalization to prevent washed-out or black outputs.
- Managing GPU memory constraints when handling multiple high-resolution frames.
- Maintaining temporal consistency across frames without recurrent modeling.

Gray Scale Video Frame



Colorized Video Frame



Fig. 4: Representative example: grayscale input (top) and colorized output (bottom).

B. Next Steps

Future improvements will focus on:

- Introducing a recurrent temporal module or optical-flow guidance to stabilize color between consecutive frames.
- Adding ESRGAN-based super-resolution to enhance fine details.
- Expanding the dataset with a broader range of scenes for improved generalization.
- Migrating the Gradio interface to a Streamlit web app for permanent deployment.

VII. RESPONSIBLE AI REFLECTION

The project adheres to responsible AI practices by using only publicly available, non-personal datasets (Kinetics-400). No facial recognition or personally identifiable data were used. The colorized outputs include a disclaimer clarifying that generated colors represent plausible estimates, not factual reconstructions. All source code and checkpoints are stored transparently within the GitHub repository to promote open, reproducible research.

REFERENCES

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” *arXiv preprint arXiv:1505.04597*, 2015. Available: <https://arxiv.org/abs/1505.04597>
- [2] CVDFoundation, “Kinetics Human Action Video Dataset,” GitHub Repository, 2024. Available: <https://github.com/cvdfoundation/kinetics-dataset>
- [3] S. Daggubati, “Video Colorization Using Deep Learning – Project Repository,” Available: <https://github.com/UF-EEE6778-Fall25/Video-Colorization/tree/main>, Accessed: Nov. 2025.